

Implementation

Database Setup

Use Database

```
mysql> USE StudentDB;
```

```
Database changed
```

1. Filtering Data (WHERE)

Query 1.1: Filter Students with Age > 20

```
mysql> SELECT * FROM Student  
-> WHERE Age > 20;
```

```
+-----+-----+-----+-----+-----+  
| StudentID | Name | Email | Age | Address |  
+-----+-----+-----+-----+-----+  
|          2 | Amit | amit@gmail.com | 21 | Mumbai |  
|          4 | Rahul | rahul@gmail.com | 22 | Nagpur |  
+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)
```

Query 1.2: Filter Students Enrolled in IT Department Courses

```
mysql> SELECT s.*  
-> FROM Student s  
-> JOIN Enrollment e ON s.StudentID = e.StudentID  
-> JOIN Course c ON e.CourseID = c.CourseID  
-> JOIN Instructor i ON c.InstructorID = i.InstructorID  
-> WHERE i.Department = 'IT';
```

```
+-----+-----+-----+-----+-----+  
| StudentID | Name | Email | Age | Address |  
+-----+-----+-----+-----+-----+  
|          1 | Ramesh Patil | ramesh@gmail.com | 20 | Pune |  
|          3 | Sneha | sneha@gmail.com | 19 | Nashik |  
+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)
```

Query 1.3: Filter Students Enrolled under Instructor ID 101

```
mysql> SELECT s.*  
-> FROM Student s  
-> JOIN Enrollment e ON s.StudentID = e.StudentID  
-> JOIN Course c ON e.CourseID = c.CourseID  
-> WHERE c.InstructorID = 101;
```

StudentID	Name	Email	Age	Address
1	Ramesh Patil	ramesh@gmail.com	20	Pune
3	Sneha	sneha@gmail.com	19	Nashik

2 rows in set (0.00 sec)

2. Sorting Data (ORDER BY)

Query 2.1: Sort Students by Name (Ascending)

```
mysql> SELECT * FROM Student
-> ORDER BY Name ASC;
```

StudentID	Name	Email	Age	Address
2	Amit	amit@gmail.com	21	Mumbai
5	Priya	priya@gmail.com	20	Delhi
4	Rahul	rahul@gmail.com	22	Nagpur
1	Ramesh Patil	ramesh@gmail.com	20	Pune
3	Sneha	sneha@gmail.com	19	Nashik

5 rows in set (0.00 sec)

Query 2.2: Sort Courses by Credits (Descending)

```
mysql> SELECT * FROM Course
-> ORDER BY Credits DESC;
```

CourseID	CourseName	Credits	InstructorID
201	DBMS	4	101
202	Data Science	3	102

2 rows in set (0.01 sec)

3. Aggregation (GROUP BY)

Query 3.1: Count Number of Students per Course

```
mysql> SELECT c.CourseName, COUNT(e.StudentID) AS Student_Count
-> FROM Course c
-> LEFT JOIN Enrollment e ON c.CourseID = e.CourseID
-> GROUP BY c.CourseName;
```

```

+-----+-----+
| CourseName | Student_Count |
+-----+-----+
| Data Science | 1 |
| DBMS | 2 |
+-----+-----+
2 rows in set (0.00 sec)

```

Query 3.2: Calculate Average Age of All Students

```

mysql> SELECT AVG(Age) AS Average_Age
-> FROM Student;

```

```

+-----+
| Average_Age |
+-----+
| 20.4000 |
+-----+
1 row in set (0.00 sec)

```

4. Nested Subqueries

Query 4.1: Find Students Enrolled in the 'DBMS' Course

```

mysql> SELECT Name FROM Student
-> WHERE StudentID IN (
->     SELECT StudentID FROM Enrollment
->     WHERE CourseID = (
->         SELECT CourseID FROM Course
->         WHERE CourseName = 'DBMS'
->     )
-> );

```

```

+-----+
| Name |
+-----+
| Ramesh Patil |
| Sneha |
+-----+
2 rows in set (0.00 sec)

```

Query 4.2: Find Students with Age Above Average

```

mysql> SELECT * FROM Student
-> WHERE Age > (
->     SELECT AVG(Age) FROM Student
-> );

```

```

+-----+-----+-----+-----+-----+
| StudentID | Name | Email | Age | Address |
+-----+-----+-----+-----+-----+

```

```

+-----+-----+-----+-----+-----+
|      2 | Amit  | amit@gmail.com |    21 | Mumbai |
|      4 | Rahul | rahul@gmail.com |    22 | Nagpur  |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

Query 4.3: Find Courses with More than 1 Student Enrolled

```

mysql> SELECT CourseName FROM Course
-> WHERE CourseID IN (
->     SELECT CourseID FROM Enrollment
->     GROUP BY CourseID
->     HAVING COUNT(StudentID) > 1
-> );

```

```

+-----+
| CourseName |
+-----+
| DBMS       |
+-----+
1 row in set (0.00 sec)

```